Density-Dependent Marked Point Process Framework (DenMark) Tutorial

Mingchi Xu, Alexandra M. Schmidt and Qihuang Zhang

October 22, 2026

Contents

0. Prepare packages and functions **2**

1. Simulate single-cell resolution ST data and select hyperparameters **4**

2. ROI selection, gene filtering and gridding **5**

3. Single-gene analysis: quantify the density-expression association **10**
3.1 Fit the full and the simplified DenMark models 10
3.2 Posterior summaries 11
3.3 Model comparison and diagnostics 12
3.4 Latent field decomposition 13
3.5 Spatial correlation functions and ranges 14

4. DCG discovery: identify the density-correlated genes **15**

5. The same workflow in Python **16**

Session information **17**

This tutorial shows how to use **DenMark** to quantify the correlation between gene expression and cell density in single-cell resolution spatial transcriptomics (ST) data, and how to identify the density-correlated genes (DCGs). All analyses use the user-level functions in `DenMark-functions.R` together with the two Stan models:

- `DenMark-M2.stan`: the full DenMark model, with dependent density and expression latent fields;
- `DenMark-M1.stan`: the simplified DenMark model, with independent fields and $a_{21} = 0$, used for model comparison.

Each model also has a variable-area version, `DenMark-M2-vararea.stan` and `DenMark-M1-vararea.stan`. These are identical except that every grid cell has its own area offset. They are used automatically when the analysis is restricted to the tissue by an alpha-shape boundary, as in this tutorial.

For a grid cell g with area $|g|$ inside the tissue, cell count $Y(g)$ and summed expression $M(g)$, the full model is

$$\begin{aligned} Y(g) &\sim \text{Poisson}\{\lambda_1(g)\}, \quad \log \lambda_1(g) = \log |g| + \beta_0 + a_{11} w_1(g), \\ M(g) \mid Y(g) &\sim \text{Poisson}\{\lambda_2(g)\}, \quad \log \lambda_2(g) = \beta_1 + \log\{Y(g) + 0.05\} + a_{21} w_1(g) + a_{22} w_2(g), \end{aligned}$$

where w_1, w_2 are independent unit-variance Matérn-3/2 Gaussian processes, approximated with Hilbert-space basis functions (HSGP). With $a_{11} = \sigma_1$, $a_{21} = \sigma_2 \rho$ and $a_{22} = \sigma_2 \sqrt{1 - \rho^2}$, the parameter ρ, named `Astar[2,1]` in Stan, is the **density-expression correlation**.

This tutorial runs on a **simulated** dataset in which the true density-expression correlations are known, so every result can be checked against the truth. The simulated data have the same structure as real single-cell resolution ST data, so the same steps apply directly to, for example, the 10x Xenium human breast cancer dataset (In Situ Sample 1, Replicate 1). A real analysis starts from two files:

- `cell_spatial.csv`: the 2D locations and cell types of the cells, one row per cell ID;
- `cell_by_gene_expression.csv`: the raw gene expression of the cells, one row per cell ID and one column per gene.

Code Block 1.2 shows how to load these two files in place of the simulated data.

Section 1 simulates the data and sets the hyperparameters. Section 2 selects the region of interest (ROI), delineates the tissue with an alpha shape, filters the genes and grids the data. Section 3 quantifies the density-expression relationship for one gene. Section 4 identifies the DCGs, and Section 5 shows the same workflow in Python.

> **Running this file.** Put `DenMark-functions.R` and the four Stan files `DenMark-M1.stan`, `DenMark-M2.stan`, `DenMark-M1-vararea.stan` and `DenMark-M2-vararea.stan` in the folder given by the `denmark_dir` parameter. A relative `denmark_dir` such as the default `"DenMark"` is read from the folder that contains this Rmd file; an absolute path such as `"C:/Users/name/Desktop/Tutorial/DenMark"` also works. If the DenMark files sit next to the Rmd file, they are found there as well. Then knit, or run `rmarkdown::render("Tutorial_DenMark.Rmd")`. Fitted models are saved in `fit_dir` and reused on the next knit. Set `refit: true` to refit.

---

0. Prepare packages and functions

DenMark runs Stan through **cmdstanr**, which needs a working CmdStan installation. Install both once, following https://mc-stan.org/cmdstanr/:

```
# [Code Block 0.1] One-time installation
install.packages("cmdstanr",
                 repos = c("https://stan-dev.r-universe.dev", getOption("repos")))
cmdstanr::check_cmdstan_toolchain(fix = TRUE)
cmdstanr::install_cmdstan(cores = 4)

# packages used by the DenMark functions and this tutorial
install.packages(c("posterior", "loo", "ggplot2", "patchwork", "knitr", "sf"))
```

Load the packages and the DenMark functions. `rstan` and `rethinking` are no longer needed.

```
# [Code Block 0.2] Load packages and DenMark functions
library(cmdstanr)
library(posterior)
library(loo)
library(ggplot2)
library(patchwork)
library(knitr)

if (nzchar(params$cmdstan_path)) set_cmdstan_path(params$cmdstan_path)
cmdstan_version()
```

```
## [1] "2.36.0"
```

```
# Locate the DenMark folder. Relative paths are read from the folder that
# contains this Rmd file, which is the working directory when knitting.
```

```
# The folder given by `denmark_dir` is tried first, then the Rmd folder itself.
denmark_files <- c("DenMark-functions.R", "DenMark-M1.stan", "DenMark-M2.stan",
                   "DenMark-M1-vararea.stan", "DenMark-M2-vararea.stan")
candidates <- unique(c(params$denmark_dir, "."))
found <- candidates[file.exists(file.path(candidates, "DenMark-functions.R"))]
if (length(found) == 0)
  stop("DenMark-functions.R was not found in: ",
       paste(normalizePath(candidates, mustWork = FALSE), collapse = " or "),
       ". Set `denmark_dir` in the YAML header to the folder with the DenMark files.")
denmark_dir <- normalizePath(found[1])
missing <- denmark_files[!file.exists(file.path(denmark_dir, denmark_files))]
if (length(missing) > 0)
  stop("Missing in ", denmark_dir, ": ", paste(missing, collapse = ", "))
if (!requireNamespace("sf", quietly = TRUE))
  stop("The tissue boundary needs the 'sf' package: install.packages('sf')")
denmark_dir
```

```
## [1] "/tmp/claude-0/pathtest/a/DenMark"
```

```
# DenMark functions; the Stan files are looked up in the same folder
source(file.path(denmark_dir, "DenMark-functions.R"))
options(denmark.stan_dir = denmark_dir)

# folder for the saved fits, created if it does not exist
fit_dir <- params$fit_dir
dir.create(fit_dir, showWarnings = FALSE, recursive = TRUE)

# a compact table printer for this document
show_table <- function(x, digits = 3, ...) {
  kable(x, digits = digits, booktabs = TRUE, row.names = FALSE, ...)
}
```

The functions used in this tutorial are:

Step	Function	Purpose
QC	`denmark_filter_genes()`	keep informative genes
Tissue boundary	`denmark_alpha_shape()`	alpha shape of all cells in the section
Gridding	`buildgridpp()`, `grid_pts_centroid()`	cell counts Y, expression M, grid geometry
Fitting	`DenMark()`	fit M2 (full) or M1 (simplified) to one gene
Summaries	`denmark_summary()`, `denmark_draws()`, `denmark_compare()`	posterior summary, ρ draws, WAIC
Interpretation	`denmark_fields()`, `denmark_decay()`	latent field decomposition, spatial correlation ranges
DCG discovery	`denmark_scan()`	fit many genes, classify PCG / NCG
Plots	`plot_denmark_boundary()`, `plot_denmark_grid()`, `plot_denmark_fields()`, `plot_denmark_decay()`, `plot_denmark_scan()`	figures

1. Simulate single-cell resolution ST data and select hyperparameters

We simulate cells from the DenMark data-generating process with `denmark_simulate()`. As in a real section, the cells lie in an irregular, lobed piece of tissue inside a 6 mm × 4 mm window, not in a rectangle. The tissue holds about 5,000 cells of the analyzed cell type and 4,000 cells of other types. The analyzed cells carry three genes with known density-expression correlations: geneA with $\rho = 0.6$, geneB with $\rho = 0$ and geneC with $\rho = -0.6$. A fourth, rarely detected gene is added to show the quality-control step.

```
# [Code Block 1.1] Simulate data with known rho
sim <- denmark_simulate(rho = c(geneA = 0.6, geneB = 0, geneC = -0.6),
                        width = 6, height = 4, n_cells = 5000, seed = 1,
                        tissue = "irregular", n_other = 4000,
                        celltype = params$celltype)
table(sim$celltype)
```

```
##
## Invasive_Tumor          Other
##           3449           4000
```

```
# arrange the simulated data in the same two tables as a real dataset
cell_spatial <- data.frame(X = seq_len(nrow(sim)) - 1, x = sim$x, y = sim$y,
                           celltype = sim$celltype)
cell_gene <- data.frame(cell_id = cell_spatial$X, sim[, c("geneA", "geneB", "geneC")])
cell_gene$lowGene <- rpois(nrow(sim), 0.01)  # rarely detected, removed by QC below
head(cell_spatial)
```

```
##   X        x        y       celltype
## 1 0 4.958866 2.517796 Invasive_Tumor
## 2 1 3.398667 1.109772 Invasive_Tumor
## 3 2 2.060565 2.566543 Invasive_Tumor
## 4 3 1.463342 2.365366 Invasive_Tumor
## 5 4 5.085220 2.566559 Invasive_Tumor
## 6 5 2.149059 1.976934 Invasive_Tumor
```

```
head(cell_gene)
```

```
##   cell_id geneA geneB geneC lowGene
## 1       0     3     1     1       0
## 2       1     2     1     4       0
## 3       2     1     3     3       0
## 4       3     3     0     4       0
## 5       4    10     1     1       0
## 6       5     0     8     1       0
```

```
gene_choice <- "geneA"
scan_genes  <- c("geneA", "geneB", "geneC")
unit_label  <- "mm"
```

Using a real dataset. To analyze the Xenium data instead, run Code Block 1.2 in place of Code Block 1.1. It creates the same objects: `cell_spatial`, `cell_gene`, `gene_choice`, `scan_genes` and `unit_label`. The rest of the tutorial is unchanged. In this document the chunk is shown but not run; set `eval = TRUE` in its header, or run it in the console, to use it. The `data_dir`, `gene` and `scan_genes` parameters in the YAML header set the data folder, the gene analyzed in Section 3 and the genes scanned in Section 4.

```
# [Code Block 1.2] Optional: load the Xenium breast cancer data
cell_gene    <- read.csv(file.path(params$data_dir, "cell_by_gene_expression.csv"),
```

```
                          check.names = FALSE)                # cell ID x genes
cell_spatial <- read.csv(file.path(params$data_dir, "cell_spatial.csv"))
                                                              # cell ID, x, y, celltype
dim(cell_gene)
colnames(cell_gene)[2:6]   # the first 5 genes

# coordinates from micrometers to millimeters
cell_spatial$x <- cell_spatial$x / 1000
cell_spatial$y <- cell_spatial$y / 1000

gene_choice <- params$gene         # e.g. "CDH1"
scan_genes  <- params$scan_genes   # e.g. CDH1, ERBB2, EPCAM, KRT8
unit_label  <- "mm"
```

The hyperparameters control the approximation precision:

- `grid_size`: number of grid cells per axis, given as a single number or as `c(nx, ny)`;
- `basis`: number of HSGP basis functions per axis. More basis functions approximate shorter length-scales better, at a higher computational cost;
- `boundfactor`: HSGP boundary factor $c \geq 1$, which removes boundary effects; $c = 1.5$ is a good default.

```
# [Code Block 1.3] Select hyperparameters
grid_size       <- params$grid_size
basis_choice    <- params$basis
boundary_choice <- params$boundfactor

# sampler settings used for every fit in this tutorial
mcmc <- list(chains = params$chains, iter_warmup = params$iter_warmup,
             iter_sampling = params$iter_sampling, adapt_delta = 0.95,
             max_treedepth = 10, seed = 2024)
c(grid_size = grid_size, basis = basis_choice, boundfactor = boundary_choice)
```

```
##   grid_size       basis boundfactor
##        10.0        12.0         1.5
```

2. ROI selection, gene filtering and gridding

The ROI is chosen to suit the research question. Here we run DenMark on all cells of one cell type across the whole tissue, which is a large-range analysis. A smaller window can be analyzed the same way by subsetting the cells, or by passing `bbox = c(xmin, xmax, ymin, ymax)` to the functions.

```
# [Code Block 2.1] Extract the cell type of interest
sel <- which(cell_spatial$celltype == params$celltype)
cells_roi <- cell_spatial[sel, ]
gene_roi  <- cell_gene[sel, -1, drop = FALSE]  # drop the cell ID column
cat(length(sel), "cells of type", params$celltype, "\n")
```

```
## 3449 cells of type Invasive_Tumor
```

```
ggplot(cells_roi, aes(x, y)) +
  geom_point(size = 0.15, alpha = 0.6) +
  coord_equal() +
  labs(x = paste0("x (", unit_label, ")"), y = paste0("y (", unit_label, ")")) +
  theme_classic()
```

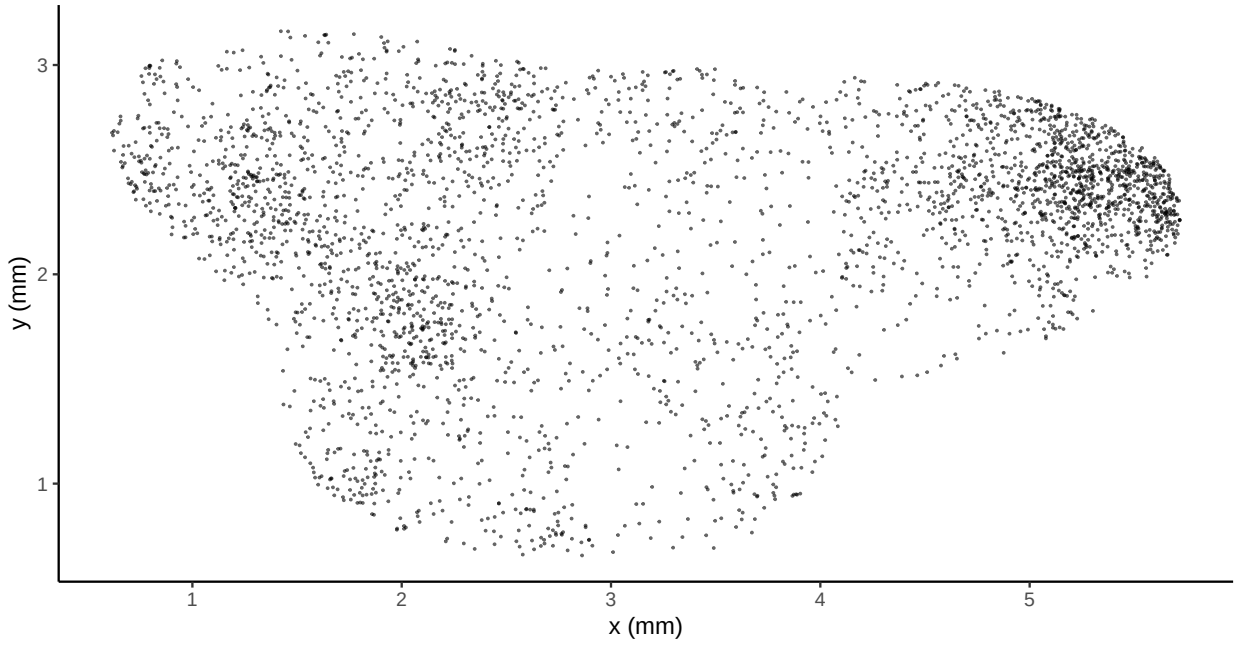


Figure 1: Cells of the selected cell type.

```
# Short-range analysis: e.g. a pathologist-selected window [0.5, 1.5] * [2, 3]
# keep <- with(cell_spatial, x >= 0.5 & x <= 1.5 & y >= 2 & y <= 3)
```

Tissue boundary. A tissue section is not a rectangle. If the grid covered the whole bounding box, grid cells outside the tissue would contain no cells simply because there is no tissue there, and the model would read them as regions of low cell density. We therefore delineate the tissue with an alpha shape. `denmark_alpha_shape()` triangulates the cell locations and keeps every Delaunay triangle whose circumradius is below α; the union of the kept triangles is the tissue. By default α is three times the median Delaunay edge length. The boundary is computed from **all** cells of the section, not only the analyzed cell type, because the region where a cell type could have been observed is the whole tissue.

```
# [Code Block 2.2] Tissue boundary by alpha shape, computed from all cells
tissue <- denmark_alpha_shape(cell_spatial$x, cell_spatial$y)
c(alpha = attr(tissue, "alpha"), tissue_area = as.numeric(sf::st_area(tissue)))
```

```
##       alpha tissue_area
##   0.1068781   8.5385318
```

```
# grid over the tissue: each grid cell gets its area inside the tissue and a type
grid_types <- buildgridpp(grid_size = grid_size,
                          dataset = data.frame(cells_roi[, c("x", "y")], n = 1),
                          boundary = tissue)
table(grid_types$grid$type)
```

```
##
## boundary exterior interior
##       36       18       46
```

```
plot_denmark_boundary(cell_spatial, tissue, grid = grid_types,
                      highlight = cell_spatial$celltype == params$celltype,
                      xlab = unit_label, ylab = unit_label)
```

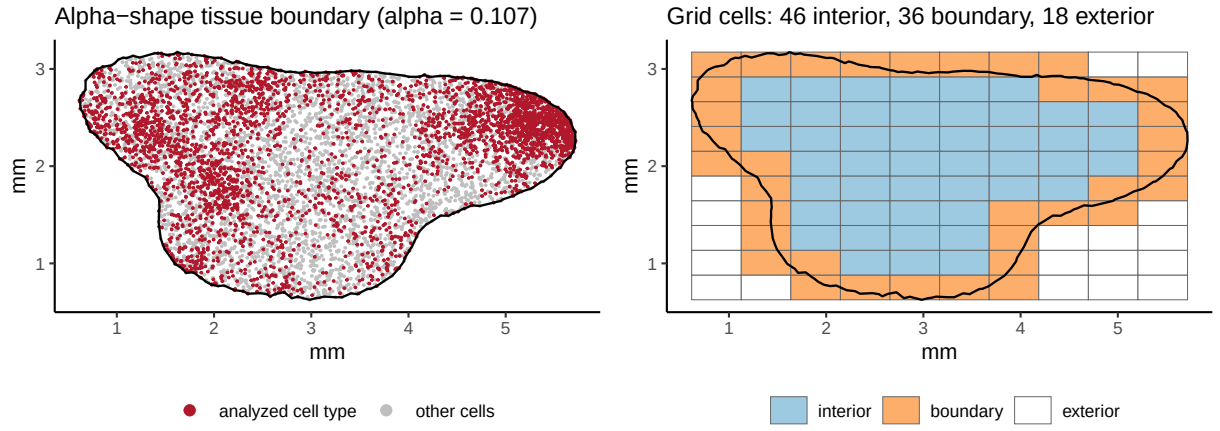


Figure 2: Left: all cells, the analyzed cell type in red, and the alpha-shape tissue boundary. Right: grid cells classified as interior, boundary or exterior.

Each grid cell is classified as **interior**, entirely inside the tissue, **boundary**, partly inside, or **exterior**, outside the tissue. Exterior grid cells are removed. Boundary grid cells are kept with their clipped area $|g \cap \text{tissue}|$, so the area offset $\log|g|$ in the model becomes a per-grid vector, and `DenMark()` switches to the variable-area Stan models. All of this happens when `boundary = tissue` is passed to the DenMark functions below. Without a `boundary`, the whole bounding box is used with a common grid area, as in the original DenMark analysis.

Quality control. Genes in the lowest 5% of variance across cells, or detected in fewer than 5% of the cells, are removed.

```
# [Code Block 2.3] Filter uninformative genes
genes_keep <- denmark_filter_genes(gene_roi, var_quantile = 0.05, min_nonzero = 0.05)
length(genes_keep)
```

```
## [1] 3
```

```
head(genes_keep, 10)
```

```
## [1] "geneA" "geneB" "geneC"
```

```
# combined data set for DenMark: one row per cell, columns x, y, genes
final_df <- data.frame(x = cells_roi$x, y = cells_roi$y,
                       gene_roi[, genes_keep, drop = FALSE], check.names = FALSE)
final_df[1:3, 1:min(6, ncol(final_df))]
```

```
##          x        y geneA geneB geneC
## 1 4.958866 2.517796     3     1     1
## 2 3.398667 1.109772     2     1     4
## 3 2.060565 2.566543     1     3     3
```

We analyze one gene in detail. The input to DenMark is a data frame with the columns `x`, `y` and the gene's counts.

```
# [Code Block 2.4] Select the gene of interest
stopifnot(gene_choice %in% names(final_df))
df_gene <- final_df[, c("x", "y", gene_choice)]

ggplot(df_gene, aes(x, y, color = .data[[gene_choice]])) +
  geom_point(size = 0.3) +
  scale_color_gradient(low = "gray92", high = "red", trans = "sqrt") +
  coord_equal() +
  labs(x = paste0("x (", unit_label, ")"), y = paste0("y (", unit_label, ")"),
       color = gene_choice) +
  theme_classic()
```

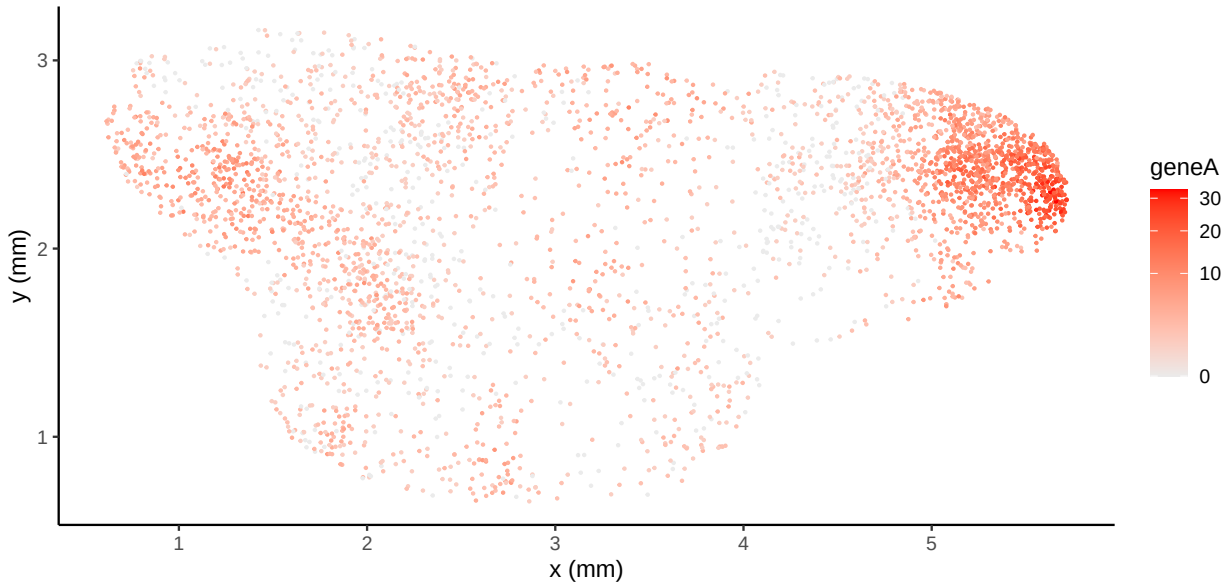


Figure 3: Cell-level expression of the selected gene.

Gridding. `buildgridpp()` assigns every cell to exactly one grid cell and returns the counts $Y(g)$ and the summed expression $M(g)$, both as vectors and as matrices. With `boundary = tissue`, it also returns the area of every grid cell inside the tissue. `grid_pts_centroid()` returns the grid geometry. DenMark keeps the grid cells inside the tissue, centers them at the origin so that the ROI becomes $[-L_1, L_1] \times [-L_2, L_2]$, and uses the grid centroids as the HSGP coordinates. This is done inside `DenMark()`, so you no longer need to compute L_1 and L_2 yourself.

```
# [Code Block 2.5] Grid the data
grid_res <- buildgridpp(grid_size = grid_size, dataset = df_gene, boundary = tissue)
names(grid_res)
```

```
##  [1] "loc_mark_mat" "Ymat"         "Mmat"         "grid"         "bbox"
##  [6] "step"         "grid_area"    "grid_size"    "gene"         "boundary"
```

```r
# grid_res[[1]]: N x 2 matrix of cell counts Y and expression M
# grid_res[[2]]: matrix of cell counts;  grid_res[[3]]: matrix of expression
head(grid_res$grid[grid_res$grid$type != "exterior", ])
```

```
##    grid_index i j        x         y  Y  M        area     type
## 3           3 3 1 1.891508 0.7530425 12 18 0.027040466 boundary
## 4           4 4 1 2.401581 0.7530425 29 63 0.092460520 boundary
## 5           5 5 1 2.911655 0.7530425 31 78 0.114683411 boundary
## 6           6 6 1 3.421729 0.7530425 14 13 0.096590579 boundary
## 7           7 7 1 3.931803 0.7530425  1  1 0.009259214 boundary
## 12         12 2 2 1.381434 1.0078334  8 15 0.012543206 boundary
```

```r
geom <- grid_pts_centroid(grid_size = grid_size, dataset = df_gene,
                          bbox = grid_res$bbox)
c(full_grid_area = geom$grid_area, n_grids = geom$N)
```

```
## full_grid_area        n_grids
##      0.1299622    100.0000000
```

```r
# the Stan input that DenMark() builds internally, useful for checks
prep <- denmark_prepare(df_gene, gene = gene_choice, grid_size = grid_size,
                        basis = basis_choice, boundfactor = boundary_choice,
                        boundary = tissue)
c(n_grids_in_tissue = prep$stan_data$n, interior = prep$info$n_interior,
  boundary = prep$info$n_boundary, L1 = prep$info$L1, L2 = prep$info$L2,
  HSGP_basis_total = prep$stan_data$mstar)
```

```
## n_grids_in_tissue          interior          boundary                L1                L2
##         82.000000         46.000000         36.000000          2.550369          1.273955
##  HSGP_basis_total
##        144.000000
```

```r
summary(exp(prep$stan_data$log_grid_area))   # clipped areas of the grid cells
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## 0.001567 0.093493 0.129962 0.104128 0.129962 0.129962
```

```r
# [Code Block 2.6] Visualize the gridded data
plot_denmark_grid(grid_res, xlab = unit_label, ylab = unit_label)
```

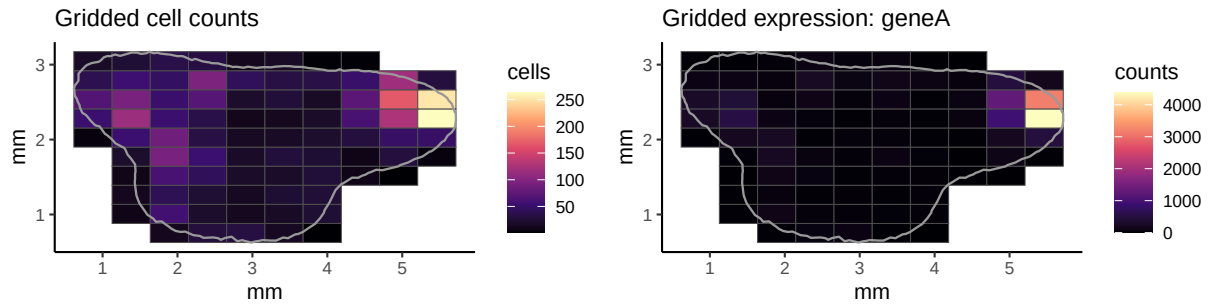


Figure 4: Gridded cell counts and gene expression in the grid cells inside the tissue.

3. Single-gene analysis: quantify the density-expression association

3.1 Fit the full and the simplified DenMark models

`DenMark()` grids the data, builds the HSGP inputs, compiles the Stan model once per session, sets valid initial values and samples. `model = "M2"` fits the full model and `model = "M1"` the simplified model. Passing `boundary = tissue` restricts the fit to the tissue. The fits are saved with `denmark_save()` so they can be reloaded without refitting.

```
# [Code Block 3.1] Run the full and simplified DenMark versions
fit_or_load <- function(file, ...) {
  if (!params$refit && file.exists(file)) return(denmark_load(file))
  fit <- do.call(DenMark, c(list(...), mcmc))
  denmark_save(fit, file)
  fit
}
tag <- paste(gene_choice, grid_size, basis_choice, params$iter_sampling, "alpha",
             sep = "_")

denmark_full <- fit_or_load(file.path(fit_dir, paste0("M2_", tag, ".rds")),
                            dataset = df_gene, gene = gene_choice, model = "M2",
                            grid_size = grid_size, basis = basis_choice,
                            boundfactor = boundary_choice, boundary = tissue,
                            quiet = TRUE)

denmark_simple <- fit_or_load(file.path(fit_dir, paste0("M1_", tag, ".rds")),
                              dataset = df_gene, gene = gene_choice, model = "M1",
                              grid_size = grid_size, basis = basis_choice,
                              boundfactor = boundary_choice, boundary = tissue,
                              quiet = TRUE)
denmark_full
```

```
## DenMark fit  |  model M2  |  gene geneA
##   grid 10 x 10 (82 grids, 3449 cells)  |  basis 12  |  boundfactor 1.50
```

```
##   rho (density-expression correlation): 0.608  [95% CrI 0.340, 0.824]
##   WAIC: 1166.42 (SE 27.80)
##   max R-hat 1.013 | min ESS bulk 421 | divergences 0 | time 1.36 min
##
##    variable  mean    sd   q2.5   q50 q97.5 rhat ess_bulk
##       beta0 5.805 0.312  5.151 5.810 6.407 1.00     1270
##       beta1 0.527 0.322 -0.186 0.558 1.107 1.00      839
##      ell[1] 0.630 0.144  0.415 0.605 0.972 1.01      709
##      ell[2] 0.567 0.180  0.319 0.534 1.044 1.01      583
##  Astar[2,1] 0.608 0.120  0.340 0.611 0.824 1.00      846
##    sigma[1] 0.883 0.178  0.631 0.849 1.348 1.00      724
##    sigma[2] 0.919 0.182  0.680 0.883 1.375 1.01      421
```

3.2 Posterior summaries

For the full model: β_0, β_1 are the intercepts of the cell density and gene expression intensities, `ell[1]`, `ell[2]` the length-scales of the two latent fields, `Astar[2,1]` $= \rho$ the density-expression correlation, and `sigma[1]`, `sigma[2]` the standard deviations of the two fields. R-hat close to 1 and large effective sample sizes (ESS) indicate convergence.

```
# [Code Block 3.2] Posterior summaries
cols <- c("variable", "mean", "sd", "q2.5", "q50", "q97.5", "rhat",
          "ess_bulk", "ess_tail")
show_table(denmark_summary(denmark_full)[, cols], caption = "Full DenMark (M2).")
```

Table 2: Full DenMark (M2).

variable	mean	sd	q2.5	q50	q97.5	rhat	ess_bulk	ess_tail
beta0	5.805	0.312	5.151	5.810	6.407	1.001	1269.525	980.352
beta1	0.527	0.322	-0.186	0.558	1.107	1.003	839.284	561.960
ell[1]	0.630	0.144	0.415	0.605	0.972	1.006	709.480	725.837
ell[2]	0.567	0.180	0.319	0.534	1.044	1.013	583.220	355.751
Astar[2,1]	0.608	0.120	0.340	0.611	0.824	1.002	845.626	771.008
sigma[1]	0.883	0.178	0.631	0.849	1.348	1.002	724.028	619.799
sigma[2]	0.919	0.182	0.680	0.883	1.375	1.012	421.175	535.537

```
show_table(denmark_summary(denmark_simple)[, cols],
           caption = "Simplified DenMark (M1): a11 and a22 are the field SDs.")
```

Table 3: Simplified DenMark (M1): a11 and a22 are the field SDs.

variable	mean	sd	q2.5	q50	q97.5	rhat	ess_bulk	ess_tail
beta0	5.879	0.297	5.305	5.876	6.466	1.000	2428.308	1183.214
beta1	0.572	0.228	0.103	0.584	1.001	1.002	1682.263	1303.201
ell[1]	0.619	0.122	0.419	0.608	0.881	1.001	1152.918	1273.358
ell[2]	0.489	0.099	0.328	0.479	0.716	1.002	1111.971	1077.368
a11	0.838	0.137	0.619	0.821	1.166	1.005	1009.681	1299.316
a22	0.794	0.114	0.618	0.779	1.059	1.001	775.370	929.628

```
# [Code Block 3.3] Posterior of rho
draws_full <- denmark_draws(denmark_full)   # rho, a11, a21, a22, ell1, ell2, ...
rho_ci <- quantile(draws_full$rho, c(0.025, 0.975))
```

```r
ggplot(draws_full, aes(rho)) +
  geom_density(fill = "gray80") +
  geom_vline(xintercept = 0, linetype = 2) +
  geom_vline(xintercept = rho_ci, color = "firebrick", linetype = 3) +
  xlim(-1, 1) +
  labs(x = expression(rho), y = "Posterior density") +
  theme_classic()
```

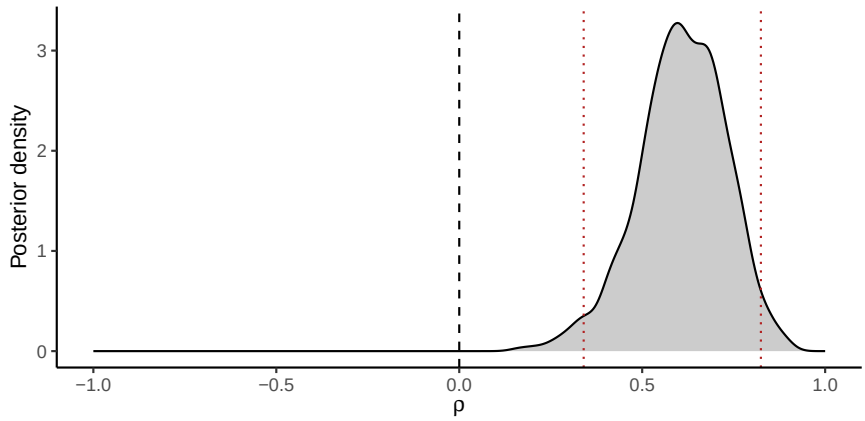


Figure 5: Posterior distribution of the density-expression correlation.

The posterior mean of ρ for geneA is **0.608** with 95% credible interval (0.340, 0.824).

3.3 Model comparison and diagnostics

```r
# [Code Block 3.4] WAIC comparison: lower WAIC is better
cmp <- denmark_compare(full_M2 = denmark_full, simplified_M1 = denmark_simple)
show_table(cmp[, c("model", "waic", "waic_se", "p_waic", "elpd_diff", "se_diff")],
           digits = 2)
```

model	waic	waic_se	p_waic	elpd_diff	se_diff
full_M2	1166.42	27.80	73.33	0.00	0.00
simplified_M1	1169.36	27.52	75.36	-1.47	2.26

```r
# sampler diagnostics and running time
diag_row <- function(f) with(f$diagnostics, data.frame(
  divergences = num_divergent, treedepth_hits = num_max_treedepth,
  max_rhat = max_rhat, min_ess_bulk = min_ess_bulk, minutes = f$time))
show_table(cbind(model = c("M2", "M1"),
                 rbind(diag_row(denmark_full), diag_row(denmark_simple))), digits = 3)
```

model	divergences	treedepth_hits	max_rhat	min_ess_bulk	minutes
M2	0	0	1.013	421.175	1.362
M1	0	0	1.005	775.370	1.250

`elpd_diff` is the difference in expected log predictive density relative to the best model, and `se_diff` is its standard error. A difference of several standard errors in favor of M2 supports a density-expression dependence.

The HSGP approximation can be checked against the rule of thumb of Riutort-Mayol et al. (2023). If the posterior length-scale is shorter than `ell_min`, increase `basis`. If `boundary_ok` is `FALSE`, increase `boundfactor`. The rule is conservative, so treat it as guidance.

```
# [Code Block 3.5] HSGP adequacy check
show_table(denmark_check_hsgp(denmark_full, warn = FALSE))
```

field	axis	ell_median	ell_min	c_needed	basis_ok	boundary_ok
density	x	0.605	1.090	1.200	FALSE	TRUE
expression	x	0.534	1.090	1.200	FALSE	TRUE
density	y	0.605	0.545	2.136	TRUE	FALSE
expression	y	0.534	0.545	1.885	FALSE	FALSE

3.4 Latent field decomposition

`denmark_fields()` returns, for every grid cell, the posterior means of the log cell density $\beta_0 + a_{11}w_1$ and the log expression per cell $\beta_1 + a_{21}w_1 + a_{22}w_2$, decomposed into the density field $a_{11}w_1$, the density-shared expression component $a_{21}w_1$, and the expression-specific component $a_{22}w_2$.

```
# [Code Block 3.6] Latent field decomposition
fields_full <- denmark_fields(denmark_full)
head(fields_full[, c("grid_index", "Y", "M", "eta1", "a11w1", "eta2", "a21w1", "a22w2")], 3)
```

```
##   grid_index  Y  M     eta1      a11w1     eta2       a21w1      a22w2
## 1          3 12 18 6.094966  0.2903828 0.5601506  0.18573844 -0.1529913
## 2          4 29 63 5.681107 -0.1234766 0.7948853 -0.07623648  0.3437184
## 3          5 31 78 5.450312 -0.3542711 0.9161762 -0.22492599  0.6136988
```

```
plot_denmark_fields(denmark_full, fields = fields_full, xlab = unit_label, ylab = unit_label)
```

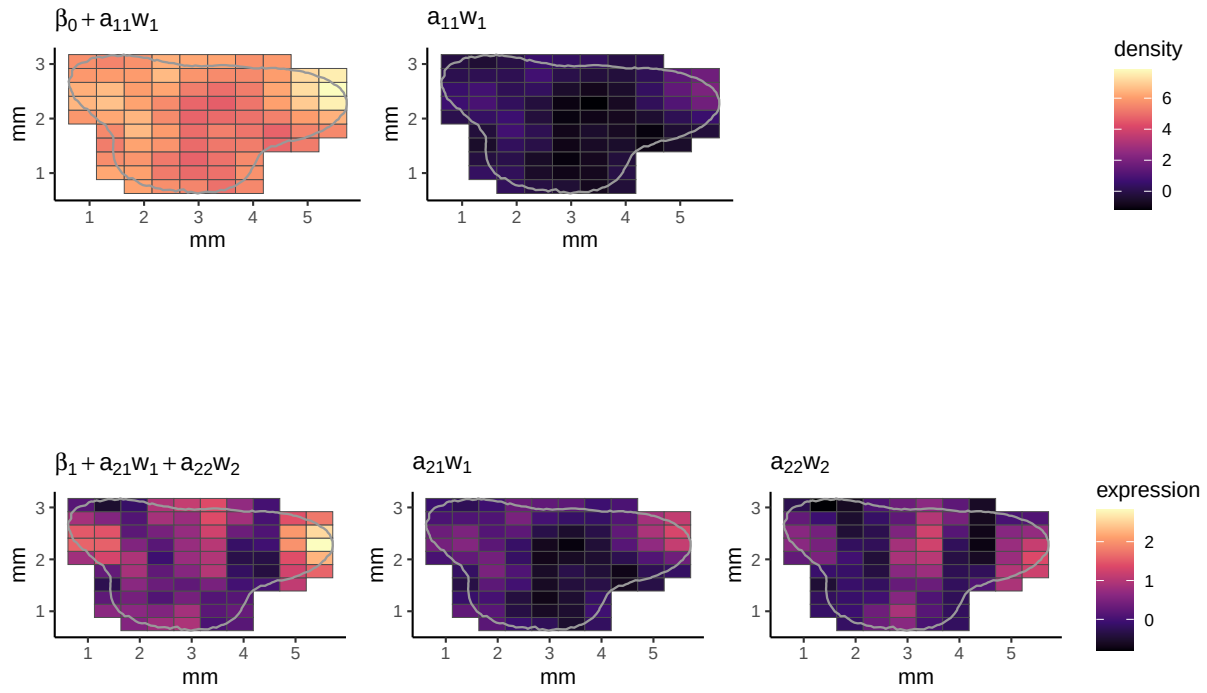


Figure 6: Latent field decomposition (posterior means). Top: cell density; bottom: gene expression.

3.5 Spatial correlation functions and ranges

The density field decays as $r_1(d) = \mathcal{M}(d; \ell_1)$. The expression field decays as $r_2(d) = \{a_{21}^2 \mathcal{M}(d; \ell_1) + a_{22}^2 \mathcal{M}(d; \ell_2)\}/(a_{21}^2 + a_{22}^2)$, where $\mathcal{M}$ is the Matérn-3/2 correlation. The practical range is the distance at which the correlation drops to 0.05. It is computed for every posterior draw.

```
# [Code Block 3.7] Spatial correlation decay
decay_full <- denmark_decay(denmark_full, threshold = 0.05)
plot_denmark_decay(denmark_full, decay = decay_full,
                   xlab = paste0("Distance (", unit_label, ")"))
```

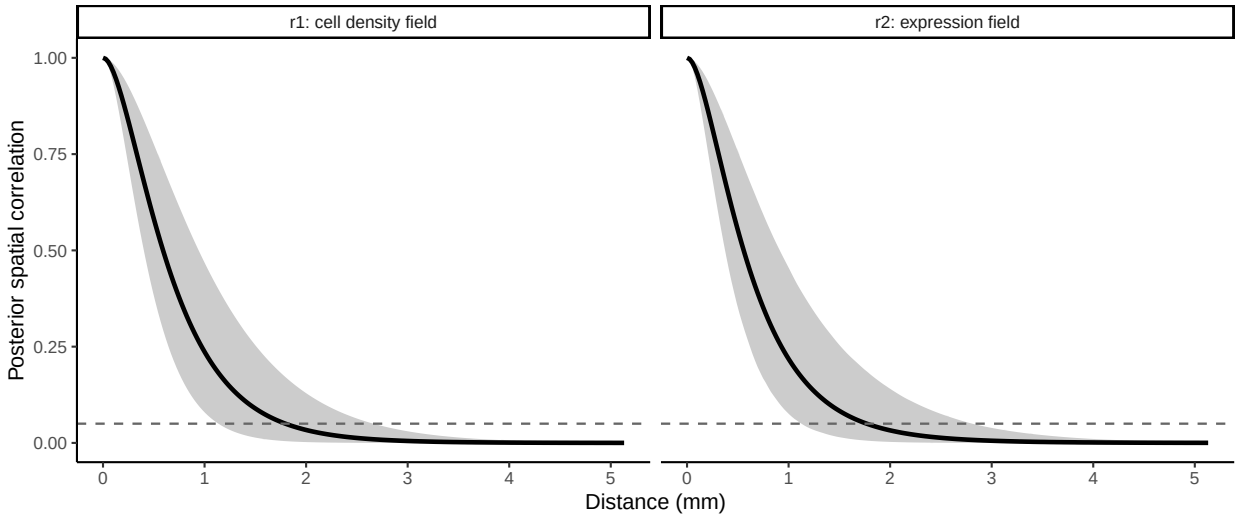


Figure 7: Posterior spatial correlation functions (mean and 95% band).

```
show_table(decay_full$ranges, caption = "Practical ranges (distance where correlation = 0.05).")
```

Table 7: Practical ranges (distance where correlation = 0.05).

field	mean	median	lower	upper
density	1.726	1.656	1.136	2.662
expression	1.705	1.635	1.126	2.808

The estimated range is 1.73 mm for cell density and 1.70 mm for the expression of geneA.

4. DCG discovery: identify the density-correlated genes

`denmark_scan()` fits the full model to each candidate gene and classifies it from the 95% credible interval of ρ:

- **PCG**, a positively density-correlated gene: the interval lies above 0;
- **NCG**, a negatively density-correlated gene: the interval lies below 0;
- **not DCG**: the interval contains 0.

Here we scan a few genes. For the whole panel, pass all filtered genes, set `save_dir` so that each gene is saved as it finishes and an interrupted scan resumes where it stopped, and consider running the genes in parallel on a cluster. `fit_simple = TRUE` also fits M1 and reports the WAIC difference per gene.

```
# [Code Block 4.1] Scan candidate genes
scan_genes <- intersect(scan_genes, names(final_df))
dcg <- do.call(denmark_scan,
               c(list(data = final_df, genes = scan_genes,
                      grid_size = grid_size, basis = basis_choice,
                      boundfactor = boundary_choice, boundary = tissue,
                      verbose = FALSE,
                      save_dir = file.path(fit_dir, paste0("scan_", tag))),
                 mcmc))
show_table(setNames(dcg[, c("gene", "rho_mean", "rho_lower", "rho_upper",
```

```
                    "class", "max_rhat")],
          c("gene", "rho", "2.5%", "97.5%", "class", "R-hat")))
```

gene	rho	2.5%	97.5%	class	R-hat
geneA	0.608	0.340	0.824	PCG	1.013
geneB	0.105	-0.249	0.449	not DCG	1.009
geneC	-0.630	-0.840	-0.384	NCG	1.007

```
# [Code Block 4.2] Forest plot of rho
plot_denmark_scan(dcg)
```

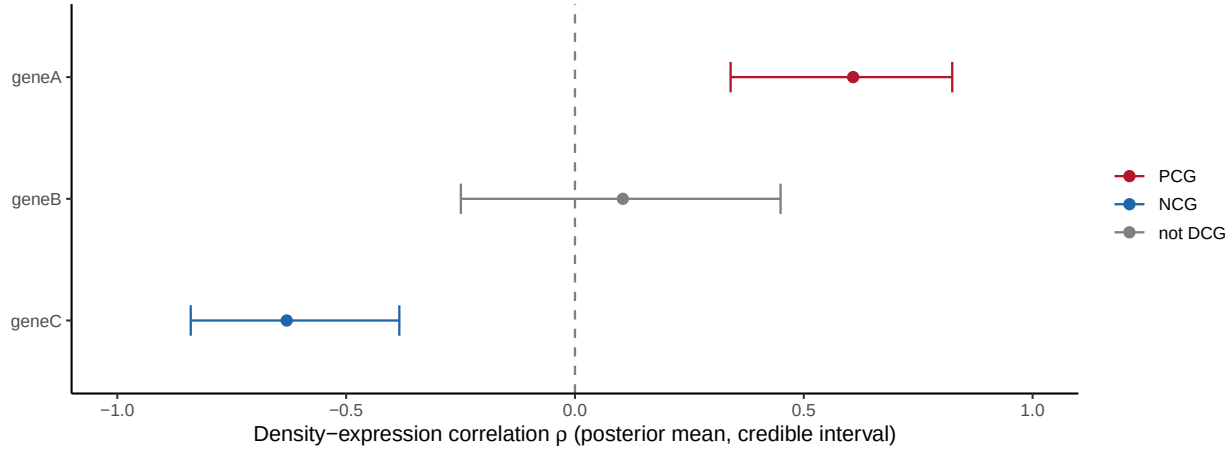


Figure 8: Density-expression correlation across genes (posterior mean and 95% credible interval).

```
# [Code Block 4.3] Full-panel scan
dcg_all <- denmark_scan(final_df, genes = genes_keep, grid_size = grid_size,
                        basis = basis_choice, boundfactor = boundary_choice,
                        boundary = tissue, save_dir = "denmark_scan_all")
table(dcg_all$class)
subset(dcg_all, class == "PCG")$gene
subset(dcg_all, class == "NCG")$gene
```

5. The same workflow in Python

`denmark.py` provides the same functions for Python, using **cmdstanpy** and the same Stan files. Given the same data, it builds identical Stan inputs and returns the same summaries. The example below runs on the simulated data; the commented lines show how to load the Xenium files instead.

```
# [Code Block 5.1] Python
# install once: pip install cmdstanpy pandas scipy matplotlib shapely
#               python -m cmdstanpy.install_cmdstan
import sys; sys.path.insert(0, "DenMark")
import pandas as pd
import denmark as dm

# simulated section: analyzed cell type plus other cells in an irregular tissue
```

```
sim = dm.simulate(rho={"geneA": 0.6, "geneB": 0.0, "geneC": -0.6},
                  width=6, height=4, n_cells=5000, seed=1,
                  tissue="irregular", n_other=4000, celltype="Invasive_Tumor")
cells = sim[["x", "y", "celltype"]]
df = sim.loc[sim["celltype"] == "Invasive_Tumor", ["x", "y", "geneA", "geneB", "geneC"]]
gene, genes_to_scan = "geneA", ["geneA", "geneB", "geneC"]

# optional: Xenium data instead
# cells = pd.read_csv("cell_spatial.csv")
# genes = pd.read_csv("cell_by_gene_expression.csv")
# cells[["x", "y"]] /= 1000                          # micrometers -> mm
# sel = cells["celltype"] == "Invasive_Tumor"
# expr = genes.loc[sel].iloc[:, 1:]
# keep = dm.filter_genes(expr)                       # QC filter
# df = pd.concat([cells.loc[sel, ["x", "y"]], expr[keep]], axis=1)
# gene, genes_to_scan = "CDH1", ["CDH1", "ERBB2", "EPCAM", "KRT8"]

# tissue boundary from ALL cells, then the grid restricted to the tissue
tissue = dm.alpha_shape(cells["x"], cells["y"])
grid = dm.buildgridpp(10, df[["x", "y", gene]], boundary=tissue)
dm.plot_boundary(cells, tissue, grid=grid,
                 highlight=cells["celltype"] == "Invasive_Tumor")

full   = dm.DenMark(df[["x", "y", gene]], gene=gene, model="M2", grid_size=10,
                    basis=25, boundfactor=1.5, boundary=tissue)
simple = dm.DenMark(df[["x", "y", gene]], gene=gene, model="M1", grid_size=10,
                    basis=25, boundfactor=1.5, boundary=tissue)
print(full)                                          # rho, WAIC, diagnostics
print(dm.compare_models(full_M2=full, simplified_M1=simple))
fields = full.fields(); decay = full.decay()
dm.plot_fields(full); dm.plot_decay(full)

dcg = dm.scan_genes(df, genes=genes_to_scan, grid_size=10, basis=25,
                    boundary=tissue, save_dir="scan_py")
dm.plot_scan(dcg)
```

Session information

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: Etc/UTC
```

```
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] knitr_1.45      patchwork_1.2.0 ggplot2_3.4.4   loo_2.6.0       posterior_1.5.0
## [6] cmdstanr_0.9.0
##
## loaded via a namespace (and not attached):
##  [1] tensorA_0.36.2.1     utf8_1.2.4           generics_0.1.3       class_7.3-22
##  [5] KernSmooth_2.23-22   lattice_0.22-5       digest_0.6.34        magrittr_2.0.3
##  [9] evaluate_0.23        grid_4.3.3           fastmap_1.1.1        Matrix_1.6-5
## [13] jsonlite_1.8.8       processx_3.8.3       e1071_1.7-14         backports_1.4.1
## [17] DBI_1.2.2            ps_1.7.6             fansi_1.0.5          viridisLite_0.4.2
## [21] scales_1.3.0         abind_1.4-5          cli_3.6.2            rlang_1.1.3
## [25] units_0.8-5          munsell_0.5.0        withr_2.5.0          yaml_2.3.8
## [29] tools_4.3.3          parallel_4.3.3       checkmate_2.3.1      dplyr_1.1.4
## [33] colorspace_2.1-0     reticulate_1.35.0    png_0.1-8            vctrs_0.6.5
## [37] R6_2.5.1             matrixStats_1.2.0    proxy_0.4-27         lifecycle_1.0.4
## [41] classInt_0.4-10      pkgconfig_2.0.3      pillar_1.9.0         gtable_0.3.4
## [45] data.table_1.17.0    glue_1.7.0           Rcpp_1.0.12          sf_1.0-15
## [49] xfun_0.41            tibble_3.2.1         tidyselect_1.2.0     farver_2.1.1
## [53] htmltools_0.5.7      labeling_0.4.3       rmarkdown_2.25       compiler_4.3.3
## [57] distributional_0.4.0
```